MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Indexes & Performance

Index Creation, Management & Query Optimization — EXPLAIN & B-Tree

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:15	Week 3 recap poll	Review
1:15 – 1:50	What are Indexes? B-Tree structure	**Theory** 📖
1:50 – 2:15	Creating & Managing Indexes	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 2:55	EXPLAIN & Query Analysis	**Theory** 📖
2:55 – 3:10	Index Best Practices & Anti-Patterns	**Theory** 📖
3:10 – 3:40	Hands-on: Indexing the University DB	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 3 Quick Recap

Concept	Key Point
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All left rows + matching right (NULLs for no match)
Self JOIN	Table joined to itself (employee→manager)
GROUP BY	Collapses rows into groups for aggregation
HAVING	Filters groups (after GROUP BY); WHERE filters rows (before)
UNION	Combines result sets vertically; removes duplicates
Subqueries	SELECT inside another statement (scalar, table, correlated)
Execution Order	FROM→JOIN→WHERE→GROUP BY→HAVING→SELECT→ORDER BY→LIMIT

📖 PART 1 — THEORY

What Are Indexes?

Why Indexes Matter

Without an index — Full Table Scan

```
Query: SELECT * FROM student WHERE last_name = 'Yılmaz';

SQLite must check EVERY row:
Row 1: last_name = 'Kaya'      ❌ skip
Row 2: last_name = 'Demir'     ❌ skip
Row 3: last_name = 'Yılmaz'    ✅ found!
Row 4: last_name = 'Çelik'     ❌ skip
...
Row 10000: last_name = 'Aksoy'  ❌ skip
```

10,000 rows → 10,000 comparisons = O(n) 🐢

With an index — Direct Lookup

```
Index on last_name → B-Tree lookup
'Yılmaz' → Row 3   ✅ found in ~14 steps
```

What Is an Index?

Analogy: Book Index

Without Index	With Index
Read every page to find "SQL"	Look up "SQL" in the back index → page 42
Linear search: O(n)	Direct lookup: O(log n)

Key Properties:

- Indexes are **separate** from the table data
- They use **additional disk space**
- They **speed up reads** (SELECT, WHERE, JOIN, ORDER BY)
- They **slow down writes** (INSERT, UPDATE, DELETE must update the index too)
- **Trade-off:** Read speed ↑ vs Write speed ↓ + Storage ↑

B-Tree Index Structure

SQLite (and most RDBMS) use **B-Tree** (Balanced Tree) indexes.

```
                    [Johnson]
                   /         \
          [Davis]              [Smith]
         /       \            /       \
    [Adams]  [Garcia]    [Lee]     [Wilson]
    [Baker]  [Harris]    [Miller]   [Young]
    [Chen]   [Jones]     [Park]     [Zhang]
```

Properties:

- **Balanced**: all leaf nodes at the same depth
- **Sorted**: values stored in order (enables range queries)
- **Logarithmic**: lookup time = O(log n)
- **Depth**: ~3–4 levels for millions of rows

Medium Post

B-Tree Implementation in PostgreSQL: Deep Dive into Database Indexing

How an Index Lookup Works

```sql
SELECT * FROM student WHERE student_id = 42;
```

Step-by-step B-Tree traversal:

```
Level 1: Root node [50] | 42 < 50 → go LEFT

Level 2: [25, 40]  | 42 > 40 → go RIGHT

Level 3: [41, 42, 43, 44] | 42 found! → points to row on disk

Total: 3 comparisons instead of scanning all rows
```

For 1 million rows:

- Full scan: **1,000,000** comparisons
- B-Tree: **~20** comparisons. **50,000x faster!**

Algorithm Visualizations

B-Trees Visual

Types of Indexes in SQLite

Index Type	Description	Created By
Primary Key Index	Automatic on `INTEGER PRIMARY KEY`	Implicit
Unique Index	Enforces uniqueness	`CREATE UNIQUE INDEX`
Regular Index	Speeds up lookups	`CREATE INDEX`
Composite Index	Multiple columns	`CREATE INDEX ... (col1, col2)`
Partial Index	Only indexes rows matching a condition	`CREATE INDEX ... WHERE condition`
Expression Index	Indexes a computed expression	`CREATE INDEX ... (expression)`

> 💡 In SQLite, `INTEGER PRIMARY KEY` is the **rowid** — it's always indexed automatically.

Creating & Managing Indexes

Creating Indexes

```sql
-- Basic index on a single column
CREATE INDEX idx_student_last_name
ON student (last_name);

-- Unique index (enforces uniqueness)
CREATE UNIQUE INDEX idx_student_email
ON student (email);

-- Composite index (multiple columns)
CREATE INDEX idx_student_dept_gpa
ON student (dept_id, gpa);

-- Index if not exists
CREATE INDEX IF NOT EXISTS idx_student_last_name
ON student (last_name);

-- Partial index (only active students)
CREATE INDEX idx_active_students
ON student (last_name)
WHERE is_active = 1;

-- Expression index
CREATE INDEX idx_student_lower_email
ON student (LOWER(email));
```

Viewing and Dropping Indexes

```sql
-- List all indexes in the database
.indexes

-- List indexes for a specific table
.indexes student

-- Show the SQL that created an index
.schema student

-- Detailed index information
PRAGMA index_list(student);

-- Columns in a specific index
PRAGMA index_info(idx_student_last_name);

-- Drop an index
DROP INDEX IF EXISTS idx_student_last_name;
```

Composite Indexes — Column Order Matters!

```sql
CREATE INDEX idx_dept_gpa ON student (dept_id, gpa);
```

This index is used for:

```sql
WHERE dept_id = 1 -- ✅ Leftmost column

WHERE dept_id = 1 AND gpa >= 3.5 -- ✅ Both columns (left to right)

WHERE dept_id = 1 ORDER BY gpa DESC -- ✅ Leftmost column + ORDER BY second
```

This index is NOT used for:

```sql
-- ❌ Skipping the leftmost column
WHERE gpa >= 3.5

-- ❌ Only the second column
ORDER BY gpa
```

🧠 Exam Focus: Composite Indexes

> **Q:** Which statement about composite indexes is TRUE?
>
> A) Column order in a composite index does not matter
>
> B) **The leftmost columns in a composite index are used first by the query optimizer** ✅
>
> C) Composite indexes are slower than single-column indexes
>
> D) SQLite does not support composite indexes

Memory Aid:

Think of a phone book sorted by **Last Name, First Name**:

- ✅ You can look up by last name → "Find all Smiths"
- ✅ You can look up by last name + first name → "Find John Smith"
- ❌ You can't look up by first name alone → "Find all Johns" (must scan everything)

EXPLAIN & Query Analysis

EXPLAIN QUERY PLAN in SQLite

```sql
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE last_name = 'Yılmaz';
```

Without index:

```
QUERY PLAN
`--SCAN student
```

> **SCAN** = full table scan (reads every row) 🐢

With index on last_name:

```
QUERY PLAN
`--SEARCH student USING INDEX idx_student_last_name (last_name=?)
```

> **SEARCH USING INDEX** = index lookup (fast!) 🚀

EXPLAIN Output Keywords

Term	Meaning	Performance
`SCAN`	Full table scan — reads every row	🔴 Slow
`SEARCH`	Uses an index to find specific rows	🟢 Fast
`USING INDEX`	Specifies which index is used	🟢 Fast
`USING COVERING INDEX`	All needed data is in the index (no table lookup)	🟢🟢 Fastest
`USING TEMP B-TREE`	Temporary sort needed (ORDER BY without index)	🟡 Medium
`AUTOMATIC COVERING INDEX`	SQLite creates a temporary index	🟡 Medium

🧠 Exam Focus: EXPLAIN

> **Q:** What type of scan does EXPLAIN show when a query uses no index?
>
> A) ref
> B) const
> C) range
> D) **ALL (full table scan)** ✅

💡 In MySQL, `ALL` = full table scan. In SQLite, `SCAN` = full table scan.

Covering Indexes

A **covering index** contains all the columns needed by a query — no table lookup required.

```sql
-- Create a covering index for this common query:
-- SELECT first_name, gpa FROM student WHERE dept_id = 1;

CREATE INDEX idx_dept_covering
ON student (dept_id, first_name, gpa);

EXPLAIN QUERY PLAN
SELECT first_name, gpa FROM student WHERE dept_id = 1;
-- SEARCH student USING COVERING INDEX idx_dept_covering (dept_id=?)
```

Why is this faster?

- Without covering index: find in index → go to table → read data
- With covering index: find in index → data already there!
- Eliminates the **table lookup** step

Index Anti-Patterns

Functions on indexed columns prevent index use!

```sql
-- ❌ Index on last_name is NOT used
SELECT * FROM student
WHERE UPPER(last_name) = 'YILMAZ';

-- ✅ Fix: use expression index
CREATE INDEX idx_upper_last ON student(UPPER(last_name));

-- ❌ Index on created_at is NOT used
SELECT * FROM events
WHERE strftime('%Y', created_at) = '2025';

-- ✅ Fix: use range comparison
SELECT * FROM events
WHERE created_at >= '2025-01-01' AND created_at < '2026-01-01';
```

🧠 Exam Focus: Index Anti-Patterns

> **Q:** Which of the following WHERE clauses will PREVENT an index on `last_name` from being used?
>
> A) `WHERE last_name = 'Smith'`
> B) `WHERE last_name LIKE 'Smi%'`
> C) **`WHERE last_name LIKE '%ith'`** ✅
> D) `WHERE last_name > 'S'`

Why?

- `= 'Smith'` → exact match → index used ✅
- `LIKE 'Smi%'` → prefix match → index used ✅
- `LIKE '%ith'` → **leading wildcard** → must scan all values ❌
- `> 'S'` → range scan → index used ✅

🧠 Exam Focus: Functions on Indexed Columns

> **Q:** A developer writes: `SELECT * FROM student WHERE YEAR(created_at) = 2024`. Why is this query slow even if `created_at` is indexed?
>
> A) The YEAR() function has a bug
>
> B) **Applying a function to an indexed column prevents the index from being used** ✅
>
> C) SELECT * causes a full table scan always
>
> D) YEAR() requires a special temporal index

💡 Functions on indexed columns = **index-breaking** anti-pattern!

When to Index (and When Not To)

✅ Index these:

- Columns used in `WHERE` clauses frequently
- Columns used in `JOIN` conditions (FK columns)
- Columns used in `ORDER BY`
- High-cardinality columns (many unique values: name, email)

❌ Don't index these:

- Small tables (full scan is fast enough)
- Low-cardinality columns (gender, boolean — few unique values)
- Columns rarely used in queries
- Tables with heavy write operations (INSERT/UPDATE/DELETE)

Index Performance Summary

Scenario	Without Index	With Index
`WHERE col = value`	O(n) full scan	O(log n) B-Tree
`WHERE col BETWEEN a AND b`	O(n) full scan	O(log n + k) range
`ORDER BY col`	O(n log n) sort	O(n) already sorted
`JOIN ON a.id = b.id`	O(n × m) nested loop	O(n log m)
`INSERT`	O(1)	O(log n) — must update index
`UPDATE indexed col`	O(1)	O(log n) — must update index

> 📝 **Exam tip:** Indexes speed up reads (SELECT) but slow down writes (INSERT/UPDATE/DELETE).

☕ BREAK (15 minutes)

PART 3 — CODING PRACTICE

Indexing the University Database

💻 Practice: Setup & Baseline

```sql
-- Open the university database
-- sqlite3 university.db
PRAGMA foreign_keys = ON;
.headers on
.mode column
.timer on    -- Show execution time!

-- Check existing indexes
.indexes

-- Check query plan WITHOUT indexes
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE last_name = 'Yılmaz';
-- SCAN student (full table scan)

EXPLAIN QUERY PLAN
SELECT * FROM student WHERE dept_id = 1 AND gpa >= 3.5;
-- SCAN student (full table scan)
```

💻 Practice: Creating Basic Indexes

```sql
-- Index on last_name
CREATE INDEX idx_student_last_name ON student(last_name);

-- Verify it's used
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE last_name = 'Yılmaz';
-- SEARCH student USING INDEX idx_student_last_name

-- Index on email (already UNIQUE, so indexed)
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE email = 'alice@ytu.edu.tr';
-- SEARCH student USING INDEX sqlite_autoindex_student_1

-- Index on foreign key
CREATE INDEX idx_student_dept ON student(dept_id);

EXPLAIN QUERY PLAN
SELECT * FROM student WHERE dept_id = 1;
-- SEARCH student USING INDEX idx_student_dept
```

💻 Practice: Composite Indexes

```sql
-- Composite index: dept_id + gpa
CREATE INDEX idx_student_dept_gpa ON student(dept_id, gpa);

-- Both columns → index used
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE dept_id = 1 AND gpa >= 3.5;
-- SEARCH student USING INDEX idx_student_dept_gpa

-- Only leftmost column → still used
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE dept_id = 1;
-- SEARCH student USING INDEX idx_student_dept_gpa

-- Only second column → NOT used!
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE gpa >= 3.5;
-- SCAN student (full table scan!)
```

💻 Practice: Covering Indexes

```sql
-- Create a covering index
CREATE INDEX idx_covering_dept
ON student(dept_id, first_name, last_name, gpa);

EXPLAIN QUERY PLAN
SELECT first_name, last_name, gpa
FROM student
WHERE dept_id = 1;
-- SEARCH student USING COVERING INDEX idx_covering_dept

-- Compare with regular index:
DROP INDEX idx_covering_dept;
EXPLAIN QUERY PLAN
SELECT first_name, last_name, gpa
FROM student
WHERE dept_id = 1;
-- SEARCH student USING INDEX idx_student_dept (then table lookup)
```

Practice: Partial Indexes

```sql
-- Only index students with GPA >= 3.0
CREATE INDEX idx_high_gpa
ON student(gpa)
WHERE gpa >= 3.0;

-- This uses the partial index
EXPLAIN QUERY PLAN
SELECT * FROM student WHERE gpa >= 3.5;
-- SEARCH student USING INDEX idx_high_gpa

-- Index for enrollment grades (only non-NULL grades)
CREATE INDEX idx_enrollment_grade
ON enrollment(grade)
WHERE grade IS NOT NULL;

EXPLAIN QUERY PLAN
SELECT * FROM enrollment WHERE grade = 'A';
-- SEARCH enrollment USING INDEX idx_enrollment_grade
```

💻 Practice: Expression Indexes

```sql
-- Index on lowercase email for case-insensitive search
CREATE INDEX idx_lower_email
ON student(LOWER(email));

EXPLAIN QUERY PLAN
SELECT * FROM student WHERE LOWER(email) = 'alice@ytu.edu.tr';
-- SEARCH student USING INDEX idx_lower_email

-- Index on birth year
CREATE INDEX idx_birth_year
ON student(strftime('%Y', birth_date));

EXPLAIN QUERY PLAN
SELECT * FROM student WHERE strftime('%Y', birth_date) = '2002';
-- SEARCH student USING INDEX idx_birth_year
```

💻 Practice: JOIN Performance

```sql
-- Without index on enrollment.student_id
EXPLAIN QUERY PLAN
SELECT s.first_name, c.course_name, e.grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id;

-- Add indexes on foreign keys
CREATE INDEX IF NOT EXISTS idx_enrollment_student ON enrollment(student_id);
CREATE INDEX IF NOT EXISTS idx_enrollment_course ON enrollment(course_id);
CREATE INDEX IF NOT EXISTS idx_course_dept ON course(dept_id);

-- Check again
EXPLAIN QUERY PLAN
SELECT s.first_name, c.course_name, e.grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id;
-- Now uses SEARCH with indexes instead of SCAN
```

💻 Practice: Analyzing Real Queries

```sql
-- Find which queries benefit from indexes

-- Query 1: Students by department and GPA
EXPLAIN QUERY PLAN
SELECT first_name, last_name, gpa
FROM student
WHERE dept_id = 2
ORDER BY gpa DESC;

-- Query 2: Enrollment count per course
EXPLAIN QUERY PLAN
SELECT c.course_name, COUNT(*)
FROM enrollment e
JOIN course c ON e.course_id = c.course_id
GROUP BY c.course_id;

-- Query 3: Students with their department (JOIN)
EXPLAIN QUERY PLAN
SELECT s.first_name, d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id
ORDER BY d.dept_name;
```

💻 Practice: Generate Large Dataset for Testing

```sql
-- Create a large test table to see real performance differences
CREATE TABLE large_test (
    id INTEGER PRIMARY KEY,
    value1 TEXT,
    value2 INTEGER,
    value3 REAL,
    created_at TEXT
);

-- Insert 100,000 rows
WITH RECURSIVE cnt(x) AS (
    SELECT 1
    UNION ALL
    SELECT x + 1 FROM cnt WHERE x < 100000
)
INSERT INTO large_test (value1, value2, value3, created_at)
SELECT
    'name_' || x,
    ABS(RANDOM() % 1000),
    ROUND(RANDOM() * 1.0 / 1000000000, 2),
    date('2020-01-01', '+' || (ABS(RANDOM() % 1825)) || ' days')
FROM cnt;

SELECT COUNT(*) FROM large_test;
```

💻 Practice: Performance Comparison

```sql
-- WITHOUT index
.timer on

EXPLAIN QUERY PLAN
SELECT * FROM large_test WHERE value2 = 500;
-- SCAN large_test

SELECT COUNT(*) FROM large_test WHERE value2 = 500;
-- Note the time

-- CREATE index
CREATE INDEX idx_large_value2 ON large_test(value2);

-- WITH index
EXPLAIN QUERY PLAN
SELECT * FROM large_test WHERE value2 = 500;
-- SEARCH large_test USING INDEX idx_large_value2

SELECT COUNT(*) FROM large_test WHERE value2 = 500;
-- Note the time - much faster!

-- Range query
EXPLAIN QUERY PLAN
SELECT * FROM large_test WHERE value2 BETWEEN 100 AND 200;
-- SEARCH using index for range

-- Cleanup
DROP TABLE large_test;
```

💻 Practice: ANALYZE Command

```sql
-- ANALYZE collects statistics about indexes
-- Helps SQLite make better query planning decisions
ANALYZE;

-- View statistics
SELECT * FROM sqlite_stat1;

-- The optimizer uses these stats to choose:
-- 1. Which index to use (if multiple available)
-- 2. Whether an index scan is faster than a table scan
-- 3. The best join order for multi-table queries

-- Run ANALYZE after:
-- - Creating new indexes
-- - Major data changes (bulk INSERT/DELETE)
-- - If queries are unexpectedly slow
```

SQLite Optimization Tips

Database-Level:

```sql
-- Write-Ahead Logging (better concurrent reads)
PRAGMA journal_mode = WAL;

-- Larger page cache (more data in memory)
PRAGMA cache_size = -20000;  -- 20MB cache

-- Synchronous mode (trade durability for speed)
PRAGMA synchronous = NORMAL;  -- (default: FULL)

-- Memory-mapped I/O
PRAGMA mmap_size = 268435456;  -- 256MB
```

Query-Level Tips:

1. Use `LIMIT` when you only need a few rows
2. Select only the columns you need (avoid `SELECT *`)
3. Use `EXISTS` instead of `IN` for subqueries with large result sets
4. Prefer `WHERE` over `HAVING` when possible (filters earlier)

Summary: Index Decision Checklist

Question	If Yes →
Is the column in frequent WHERE clauses?	✅ Create index
Is it a foreign key used in JOINs?	✅ Create index
Is it used in ORDER BY?	✅ Consider index
Does the table have > 1000 rows?	✅ Index helps
Is the column high-cardinality?	✅ Index effective
Is the table write-heavy?	⚠️ Fewer indexes
Is it low-cardinality (boolean, gender)?	❌ Skip index
Is the table small (< 100 rows)?	❌ Skip index

📝 Practice Exercises (Homework)

1. **On the e-commerce database:**
   - Identify which columns should be indexed
   - Create appropriate indexes for order_item and orders tables
   - Use EXPLAIN QUERY PLAN to verify indexes are used
2. **Performance experiment:**
   - Create a table with 50,000+ rows
   - Time a query without an index, then with an index
   - Try different index types (single, composite, partial)
3. **Anti-patterns:** Write three queries that BREAK index usage (function on column, leading wildcard, type mismatch) and show how to fix each.

🔮 Next Week Preview

Week 5: Views

- What are views and why use them
- Creating and querying views
- Updatable views
- WITH CHECK OPTION
- View use cases: security & complexity hiding

Preparation:

- 📖 Read: *Learning SQL* Ch. 14 (Views)
- 💻 Complete this week's homework
- 🎓 Watch: CS50 SQL Week 4 — Viewing

Thank You!

Questions?

Next class: Views — virtual tables for security & simplicity

📧 Contact: Course LMS / Email

📚 *Learning SQL* (Beaulieu, O'Reilly)

🎓 CS50 SQL — cs50.harvard.edu/sql